Istruzioni: Scrivere immediatamente su ogni foglio cognome, nome e numero di matricola. È consentito utilizzare esclusivamente i fogli forniti dalla commissione. Tutto il materiale personale (quaderni, appunti, dispositivi elettronici), deve essere riposto nello zaino - che deve rimanere chiuso - o depositato sulla cattedra. I dispositivi elettronici (telefoni, smartwatch, fitness band, auricolari, tablet, ecc.) atti alla ricezione di informazioni devono essere spenti e non devono essere mai visibili durante la prova. La prova ha una durata di **3 ore**. Ogni violazione delle regole comporterà l'immediato annullamento della prova.

Esercizio 1 (Testing)

Il metodo statico `calculatePrize` della classe `SwEngGames` viene utilizzato in una università di Seoul per calcolare il montepremi finale da destinare ai singoli studenti che superano tutte le dure prove predisposte nell'ambito di un gioco a premi a tema Ingegneria del Software. Il metodo accetta i seguenti argomenti:

- `int nEliminated`: numero (non negativo) di studenti eliminati durante le prove;
- `int nWinners`: numero (non negativo) di studenti che hanno superato tutte le prove (vincitori);
- `String gameEdition`: stringa non `null` che indica che tipo di edizione del gioco a premi si sta considerando. Il parametro può assumere i seguenti valori: "XMAS" (versione natalizia), "EASTER" (versione pasquale), "DEFAULT" (versione base). Qualsiasi altra stringa diversa da `null` deve essere interpretata come "DEFAULT".

Se almeno uno dei parametri è non valido, il metodo solleva una `IllegalArgumentException`. In caso contrario, il metodo ritorna un numero float rappresentante il montepremi (in Won, la valuta ufficiale della Corea del Sud) che sarà destinato a ciascuno studente vincitore. Tale budget è calcolato come descritto di seguito. Innanzitutto, si calcola il budget complessivo, utilizzando la formula:

$$\text{overallBudget} = (\text{nEliminated} \times \text{coeffPrize})$$

Il coefficiente coeffPrize viene determinato in base all'edizione del gioco a premi, come da tabella seguente:

coeffPrize		
XMAS	**EASTER**	**DEFAULT**
10.000.000	2.000.000	1.000.000

Infine, il budget complessivo viene diviso in parti uguali tra tutti gli studenti che hanno superato tutte le prove. Per esempio, una invocazione del metodo con 10 studenti eliminati durante le prove, 2 studenti che hanno superato le prove e l'edizione del gioco "DEFAULT", risulterà in un budget complessivo di 10 milioni di Won, da suddividere tra due vincitori, portando a un risultato di 5 milioni di Won per studente vincitore.

a. Indicare, per ciascuno dei parametri del metodo `calculatePrize`, un partizionamento in classi di equivalenza per testing black-box;
b. Scrivere quattro test JUnit con strategia Black Box per il metodo `calculatePrize`, indicando per ciascuno di essi quali classi di equivalenza copre. Si richiede inoltre che un test corrisponda a scenari in cui almeno uno dei parametri è non valido, e che i restanti tre corrispondano a scenari in cui tutti i parametri sono validi. Ove possibile, i test devono coprire classi di equivalenza distinte.

Quanti test sono necessari per testare il metodo con strategia R-WECT? Motivare la risposta, descrivendo brevemente le diverse strategie.

facsimile

Esercizio 2 (Refactoring)

```java
// UserMessage.java

public class UserMessage {
  public String bld_Msg(String name, int points, boolean html) {
    String level;
    if (points < 50) level = "basic";
    else if (points < 200) level = "pro";
    else level = "elite";

    // bonus: 10% se pro, 25% se elite
    double bonus = 0.0;
    if (level.equals("pro")) bonus = points * 0.10;
    if (level.equals("elite")) bonus = points * 0.25;

    // messaggio
    String msg = "User " + name + " | level=" + level + " | points=" + points +
                 " | bonus=" + bonus;

    if (html) {
        // Duplicazione contenuto in HTML
        return "<strong>User</strong> " + name + " | <em>level</em>=" + level +
               " | <em>points</em>=" + points + " | <em>bonus</em>=" + bonus;
    }
    return msg;
  }
}
```

Si consideri il frammento di codice Java riportato sopra, e si svolgano i seguenti punti:

- **a)** Sono presenti dei *code smell*? Se sì, indicare quali. Per ciascun code smell individuato, fornire una breve descrizione degli svantaggi che comporta (impatti su leggibilità, manutenibilità, rischio di errori, difficoltà di test/estensione).
- **b)** Per ciascun code smell individuato, indicare le operazioni di refactoring applicabili.
- **c)** Riportare il codice rifattorizzato, dopo aver applicato le operazioni di refactoring descritte al punto precedente.

Esercizio 3

Si vuole progettare un sistema informativo per un rifugio di gattini che gestisce adozioni, cure e interazioni con i visitatori. Si descrivano almeno 5 user story, seguendo il formato standard (As a, I want, So that).